Hotel Costa Azul — Architecture Decision Records

Nombre(s): Ivan Francisco Padron Macias María Ximena Rodarte Cuevas Angel Haziel Bernal Garza	
Matrícula(s): 2150830 2154111 2156444	
Nombre del curso: Componentes en Lógica de Negocios	
Nombre del profesor: Miguel Ángel Gómez Marroquin	
Módulo / Actividad: Unidad 3 · Diseño arquitectural del sistema con IA (Hotel Costa Azul)	12/10/2026

Bibliografía / fuentes: Actividad práctica Unidad 3 «Diseño arquitectural del sistema con IA» (Hotel Costa Azul), análisis de negocio de la Unidad 2 (BRD, BPMN, glosario y User Stories) y bitácora de IA del equipo.

ADR-001 — Selección del patrón de lógica de negocio: Domain Model

- **Estado:** Aceptada
- **Fecha:** 12/10/2026
- **Contexto:** El Sistema de Gestión Hotelera (PMS) del Hotel Costa Azul administra reservas, habitaciones, check-in/check-out, folios, cargos y pagos. Las reglas de tarifas, cancelación, no-show y autorización interactúan entre conceptos y los estados cambian durante el ciclo de vida de la reserva y de la estancia.

Evidencia de Unidad 2

- **BRD (RN-01 a RN-13):** reglas complejas y acopladas sobre tarifas, horarios, garantías y roles.
- **BRD:** necesidad de adaptar la operación a la disponibilidad de canales y a cambios en políticas (p. ej. penalización por cancelación).
- **BPMN y glosario:** el ciclo de vida del huésped cruza Reservaciones, Recepción, Limpieza y Administración.
- **User Stories (US-06, US-13, US-18, US-19, US-23):** control de estados críticos (habitación Fuera de Servicio, folios abiertos), excepciones, overbooking, no-shows y límites de autorización gerencial.

Alternativas

Alternativa	Ventajas	Riesgos / consecuencias
Transaction Script	Simple y directo para flujos lineales pequeños.	Tarifas por temporada, incrementos de fin de semana y penalizaciones se repetirían en varios procedimientos; un cambio de política obliga a modificar múltiples scripts y aumenta el riesgo de errores de cobro.
Table Module	Adecuado para consultas y reportes masivos sobre estructuras tabulares.	Una habitación no es solo un registro: tiene estados (Disponible, Sucia, Limpia, Inspeccionada, OOO) y reglas de ciclo de vida que el patrón no gobierna bien; limitado para transferir cargos o manejar reservas grupales.
Domain Model	Encapsula reglas, estados, invariantes y eventos cerca de los conceptos del negocio; los cambios de política se localizan en la entidad o Value Object.	Mayor curva de aprendizaje, mayor trabajo inicial de modelado, persistencia y pruebas de dominio.

Decisión

Adoptar **Domain Model** como patrón principal. Las entidades `Reserva`, `Folio` y `Habitación` serán responsables de su ciclo de vida y de sus invariantes; los Domain Services manejarán comportamientos que cruzan más de una entidad (cálculo de tarifas y overbooking).

Las reglas de autorización y excepciones (ajustes mayores a USD 50, modificación de tarifas confirmadas, habilitación de habitaciones OOO) forman parte natural del comportamiento del modelo y no se dispersan en condicionales de procedimientos.

Consecuencias

Positivas: reglas localizadas, menor duplicidad (RN-04, RN-05), mejor trazabilidad y facilidad para emitir eventos de dominio.

Costos: curva de aprendizaje y diseño más compleja; más clases e interfaces de persistencia; mayor esfuerzo inicial que un enfoque procedural.

Trazabilidad

RN-01 a RN-13; US-06, US-13, US-18, US-19, US-23; CU-01 a CU-05.

ADR-002 — `Reserva` y `Folio` como Aggregate Roots; `Habitación` referenciada por ID

- **Estado:** Aceptada
- **Fecha:** 12/10/2026
- **Contexto:** El negocio necesita dos unidades de consistencia distintas: la intención de hospedaje antes de la llegada (disponibilidad, tarifa, cancelación, no-show) y la estancia activa (cargos, saldo, cambio de habitación y check-out). Confirmar una reserva exige evitar overbooking entre canales.

Alternativas

1. Un único agregado `Huésped` que contenga reservas, folios y habitaciones.
2. Convertir `Habitación` en Aggregate Root independiente desde la primera fase.
3. **Dos agregados: `Reserva` (raíz `Reserva`) y `EstanciaFolio` (raíz `Folio`); `Habitación` como entidad referenciada por ID y `Huésped` como entidad de apoyo.**

Decisión

Se adopta la alternativa 3.

- **Agregado Reserva:** protege la no duplicidad de reservas confirmadas en las mismas fechas, la aplicación consistente de tarifas, la penalización de la primera noche si se cancela con menos de 48 horas, el cobro automático de no-show a las 11:59 p.m. con garantía y la tarifa confirmada que no puede modificarse sin autorización gerencial (RN-04, RN-05, RN-06, RN-08).
- **Agregado EstanciaFolio:** protege que ningún cargo se agregue sin los datos obligatorios del vale (RN-12), que el folio no cierre con saldo pendiente (US-13), que los ajustes mayores a USD 50 requieran autorización previa (RN-10), la sincronización de la habitación al cambiarla y un solo folio activo por habitación.
- **Habitación** no es raíz: sus cambios de estado derivan siempre de una `Reserva` (al confirmar) o de un `Folio` (al hacer check-out); ningún caso de uso la modifica de forma aislada.
- **Huésped** se modela como entidad de apoyo; si en el futuro se implementa un CRM se promoverá a su propio bounded context.

El límite transaccional lo define el `UnitOfWork`. La garantía contra overbooking requiere además un mecanismo de concurrencia en infraestructura (restricción/bloqueo transaccional), porque la invariante cruza solicitudes simultáneas de distintos canales.

Consecuencias

- Límites de consistencia pequeños; se evita cargar un agregado gigante con todo el historial del huésped.
- Las referencias entre agregados se hacen por identidad, lo que exige coordinación mediante eventos de dominio (`ReservaConfirmada`, `CheckInRegistrado`).
- Los cambios de estado de la habitación dependen de la reserva o del folio, lo que añade coordinación entre contextos.

Trazabilidad

RN-03, RN-04, RN-05, RN-06, RN-07, RN-08, RN-10, RN-12, RN-13; US-02, US-04, US-05, US-06, US-07, US-10, US-11, US-13, US-25; CU-02, CU-03, CU-04.

ADR-003 — Domain Services y bounded contexts: `TariffCalculationService` y `WalkService`

- **Estado:** Aceptada
- **Fecha:** 12/10/2026
- **Contexto:** Existen comportamientos que no pertenecen naturalmente a una sola entidad: el cálculo de la tarifa final (BAR, temporada, fin de semana, tipo de cliente) y la reubicación por overbooking (walk) con hotel aliado, cobertura de diferencia tarifaria, transporte y cortesías (RN-06).

Alternativas

- Calcular tarifas dentro del agregado `Reserva`: menos piezas, pero la entidad acumularía dependencias externas y reglas complejas.
- Ubicar `WalkService` en el bounded context de Reservaciones: la reserva original solo cambia de estado a «Reubicada».
- **Domain Services independientes; `WalkService` en el bounded context de Estancia.**

Decisión

`TariffCalculationService` se mantiene como Domain Service independiente, sin estado, que centraliza la lógica de temporadas, canales y tipos de cliente (RN-13) y protege el principio de Responsabilidad Única.

`WalkService` se ubica en el bounded context de **Estancia**, porque la operación requiere gestionar la habitación física, abrir folio en el hotel aliado y registrar cargos de cortesía. La reserva original pasa a «Reubicada» mediante un evento.

Consecuencias

- Nuevas reglas de temporada o canal se incorporan sin modificar las entidades de dominio.
- Se separan claramente los significados de «Habitación» en cada bounded context.
- Se requiere coordinación por eventos entre Reservaciones y Estancia.

Trazabilidad

RN-06, RN-13; US-02, US-03, US-23; CU-01, CU-05.

ADR-004 — Service Layer delgada, Repositories por agregado y Unit of Work

- **Estado:** Aceptada
- **Fecha:** 12/10/2026
- **Contexto:** Se necesita un punto de entrada claro para cada caso de uso que coordine el flujo sin absorber reglas de negocio, y aislar el dominio de SQL, ORM y detalles de infraestructura.

Alternativas

- Un único servicio grande con todas las operaciones del hotel (Fat Service).
- Acceso directo del dominio a la base de datos o a un ORM.
- **Application Services por área (`ReservacionAppService`, `RecepcionAppService`), repositorios separados por agregado y `UnitOfWork` como límite transaccional.**

Decisión

Los Application Services solo orquestan: reciben la solicitud, obtienen agregados mediante repositorios, invocan el comportamiento del dominio, persisten y cierran la transacción. Métodos: `consultarYCalcularTarifa`, `confirmarReservacion`, `registrarCheckIn`, `registrarCargo`, `registrarCheckOut` y `gestionarExcepcionOverbooking`.

Se definen `ReservaRepository`, `HabitacionRepository` y `FolioRepository` como interfaces con lenguaje del negocio (`findDisponibles`, `findActiveByHabitacion`), sin SQL ni tablas. Los errores se expresan en lenguaje ubicuo (p. ej. «El folio no puede cerrarse por saldos pendientes»).

Se confirma la revisión SOLID: responsabilidad única (S), cálculo de tarifas extensible (O), implementaciones sustituibles (L), repositorios segregados (I) y dependencia de abstracciones (D).

Consecuencias

- Servicios fáciles de probar con repositorios en memoria y sustituibles sin afectar el dominio.
- Mayor número de interfaces y trabajo inicial de implementación.
- Cualquier regla que aparezca en un servicio de aplicación debe moverse al dominio.

Trazabilidad

RN-06, RN-12; US-07, US-10, US-13; CU-01 a CU-05.

ADR-005 — CQRS básico con Commands, Queries y DTOs de lectura

- **Estado:** Aceptada
- **Fecha:** 12/10/2026
- **Contexto:** El hotel necesita operaciones que cambian el estado (confirmar reserva, check-in) y consultas frecuentes (disponibilidad con tarifa cotizada, estado de cuenta del folio) con necesidades de optimización distintas.

Alternativas

- Interfaz única de servicio para lecturas y escrituras: menos piezas, pero mezcla responsabilidades.
- CQRS completo con bases separadas o event sourcing: innecesario para el alcance y la evidencia disponible.
- **CQRS básico en las operaciones principales, sin aplicarlo a operaciones auxiliares de grano fino.**

Decisión

Se aplica CQRS básico con dos Commands y dos Queries:

Tipo	Nombre	Handler	Resultado
Command	`ConfirmarReservacionCommand`	`ConfirmarReservacionHandler`	Confirmación, identificador y estado actualizado; emite `ReservaConfirmada`.
Command	`RegistrarCheckInCommand`	`RegistrarCheckInHandler`	Check-in registrado y folio de estancia abierto.
Query	`ConsultarDisponibilidadQuery`	`ConsultarDisponibilidadHandler`	`List<HabitacionDisponibleDTO>` con tarifas cotizadas.
Query	`ObtenerEstadoFolioQuery`	`ObtenerEstadoFolioHandler`	`FolioSummaryDTO` con cargos, consumos y saldo pendiente.

Los DTOs de lectura no se reutilizan como modelos de escritura. No se aplica CQRS estricto a consultas internas de apoyo entre entidades del mismo agregado ni a validaciones simples de dominio, para evitar sobreingeniería.

Consecuencias

- Lecturas optimizadas que no cargan ni bloquean los agregados de escritura.
- Más handlers y DTOs que una arquitectura CRUD simple.
- No se introduce consistencia eventual ni infraestructura de mensajería obligatoria.

Trazabilidad

RN-06, RN-12; US-02, US-03, US-07, US-13, US-25; CU-01, CU-02, CU-03, CU-04.

ADR-006 — Autorización gerencial dividida entre aplicación y dominio

- **Estado:** Aceptada
- **Fecha:** 12/10/2026
- **Contexto:** Las reglas RN-08 y RN-10 exigen autorización gerencial para descuentos o ajustes mayores a USD 50 y para modificar tarifas confirmadas, lo que genera duda sobre si es una regla de acceso (aplicación) o una regla de negocio (dominio).

Alternativas

- Resolver toda la autorización en la capa de aplicación o seguridad.
- Resolver toda la autorización dentro del dominio, incluida la validación de roles.
- **Dividir responsabilidades: la aplicación valida el rol; el dominio valida el límite y el registro de la autorización.**

Decisión

La capa de aplicación valida **quién** puede invocar la operación (rol de Recepción o Gerencia). El agregado (`Reserva` o `Folio`) mantiene como invariante el límite de USD 50 y el registro del usuario que autorizó, y rechaza ajustes mayores si no van acompañados de una autorización válida.

Consecuencias

- La regla de negocio no puede eludirse aunque se omita una validación de seguridad.
- Se conserva trazabilidad del usuario que autorizó cada cambio.
- El límite de USD 50 podría requerir configuración si la política cambia.

Trazabilidad

RN-08, RN-10; US-18, US-19, US-23.